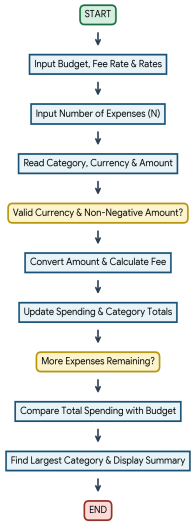
PYTHON PROGRAMMING CASE STUDIES

10 CASE STUDIES — COMPLETE ASSIGNMENT WITH FLOWCHARTS

CASE STUDY 9: MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. PROBLEM STATEMENT

Develop a Python program that converts travel expenses from different currencies into a home currency using supplied exchange rates. Apply a conversion fee, compare total spending with the planned budget, and identify the largest expense category.



2. OBJECTIVES

- Store expense amounts, currencies, and categories.
- Convert foreign-currency expenses into home currency.
- Calculate conversion fees and total expenses.
- Compare total spending with travel budget.
- Identify highest expense category.

3. ASSUMPTIONS

- The program uses illustrative exchange rates.
- The conversion fee is a percentage of converted amount.
- Expense amounts are non-negative.
- Each expense has one category and currency.

4. ALGORITHM

1. Read budget and fee percentage.
2. Read total number of expenses.
3. For each expense, read category, currency, and amount.
4. Look up exchange rate and calculate converted amount.
5. Calculate conversion fee and total expense.
6. Update spending totals and identify largest category.

5. PSEUDOCODE

```
BEGIN
  INPUT budget, fee_rate
  SET rates = {'USD': 83.0, 'EUR': 90.0, 'GBP': 105.0}
  SET total = 0, category_totals = {}
  INPUT n
  REPEAT n times:
    INPUT category, currency, amount
    IF amount >= 0 AND currency IN rates:
      converted = amount * rates[currency]
      fee = converted * fee_rate / 100
      cost = converted + fee
      total += cost
      category_totals[category] += cost
  DISPLAY total, remaining/over budget, largest category
END
```

6. PYTHON SOURCE CODE

```
rates = {"USD": 83.0, "EUR": 90.0, "GBP": 105.0}
budget = float(input("Enter travel budget in INR: "))
fee_rate = float(input("Enter conversion fee percentage: "))
n = int(input("Number of expenses: "))
total = 0.0
category_totals = {}

for i in range(n):
    print(f"\nExpense {i + 1}")
    category = input("Category: ").strip().title()
    currency = input("Currency (USD/EUR/GBP): ").strip().upper()
    amount = float(input("Amount in foreign currency: "))

    if amount < 0 or currency not in rates:
        print("Invalid amount or unsupported currency; expense skipped.")
        continue

    converted = amount * rates[currency]
    fee = converted * fee_rate / 100
    cost = converted + fee
    total += cost
    category_totals[category] = category_totals.get(category, 0) + cost
    print(f"Converted: INR {converted:.2f}")
    print(f"Fee: INR {fee:.2f}")
    print(f"Expense total: INR {cost:.2f}")

print("\n----- TRAVEL SUMMARY -----")
print(f"Total spending: INR {total:.2f}")
if total > budget:
    print(f"Over budget by: INR {total - budget:.2f}")
else:
    print(f"Budget remaining: INR {budget - total:.2f}")

if category_totals:
    largest = max(category_totals, key=category_totals.get)
    print(f"Largest expense category: {largest} (INR {category_totals[largest]:.2f})")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Enter travel budget in INR: 20000
Enter conversion fee percentage: 2
Number of expenses: 2

Expense 1
Category: Hotel
Currency (USD/EUR/GBP): USD
Amount in foreign currency: 100
Converted: INR 8300.00
Fee: INR 166.00
Expense total: INR 8466.00

Expense 2
Category: Food
Currency (USD/EUR/GBP): EUR
Amount in foreign currency: 50
Converted: INR 4500.00
Fee: INR 90.00
Expense total: INR 4590.00

----- TRAVEL SUMMARY -----
Total spending: INR 13056.00
Budget remaining: INR 6944.00
Largest expense category: Hotel (INR 8466.00)
```

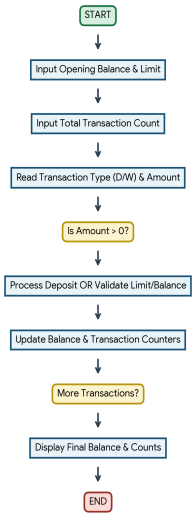
8. CONCLUSION

The program converts foreign expenses using exchange rates, applies conversion fees, tracks category spending, and verifies budget adherence.

CASE STUDY 13: ATM TRANSACTION SEQUENCE SIMULATION

1. PROBLEM STATEMENT

Create a Python program that processes a sequence of deposits and withdrawals, updates an account balance, enforces a withdrawal limit, rejects invalid transactions, and prints a transaction summary.



2. OBJECTIVES

- Simulate deposits and withdrawals.
- Maintain correct balance after every valid transaction.
- Reject negative/zero transaction amounts.
- Prevent withdrawals exceeding balance or daily limit.
- Track successful and rejected transactions count.

3. ASSUMPTIONS

- Opening balance is non-negative.
- Deposit/withdrawal amounts must be strictly positive.
- Per-transaction withdrawal limit is enforced.
- No interest or bank fees applied.

4. ALGORITHM

1. Read opening balance and maximum withdrawal limit.
2. Read total transaction count.
3. For each transaction, validate type (D/W) and amount.
4. Process deposits by adding to balance.
5. Process withdrawals after checking balance and limits.
6. Output transaction summary statistics.

5. PSEUDOCODE

```
BEGIN
  INPUT balance, withdrawal_limit, n
  SET successful = 0, rejected = 0
  REPEAT n times:
    INPUT type, amount
    IF amount <= 0: rejected++
    ELSE IF type == 'D': balance += amount; successful++
    ELSE IF type == 'W':
      IF amount <= balance AND amount <= withdrawal_limit:
        balance -= amount; successful++
      ELSE: rejected++
  DISPLAY balance, successful, rejected
END
```

6. PYTHON SOURCE CODE

```
balance = float(input("Opening balance: "))
withdrawal_limit = float(input("Maximum withdrawal per transaction: "))
n = int(input("Number of transactions: "))
successful = 0
rejected = 0

for i in range(n):
    print(f"\nTransaction {i + 1}")
    kind = input("Type (D for deposit, W for withdrawal): ").strip().upper()
    amount = float(input("Amount: "))

    if amount <= 0:
        print("Rejected: amount must be positive.")
        rejected += 1
    elif kind == "D":
        balance += amount
        successful += 1
        print(f"Deposit successful. Balance: {balance:.2f}")
    elif kind == "W":
        if amount > balance:
            print("Rejected: insufficient balance.")
            rejected += 1
        elif amount > withdrawal_limit:
            print("Rejected: withdrawal limit exceeded.")
            rejected += 1
        else:
            balance -= amount
            successful += 1
            print(f"Withdrawal successful. Balance: {balance:.2f}")
    else:
        print("Rejected: invalid transaction type.")
        rejected += 1

print("\n----- TRANSACTION SUMMARY -----")
print(f"Final balance: {balance:.2f}")
print(f"Successful transactions: {successful}")
print(f"Rejected transactions: {rejected}")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Opening balance: 1000
Maximum withdrawal per transaction: 500
Number of transactions: 3

Transaction 1
Type (D for deposit, W for withdrawal): D
Amount: 200
Deposit successful. Balance: 1200.00

Transaction 2
Type (D for deposit, W for withdrawal): W
Amount: 300
Withdrawal successful. Balance: 900.00

Transaction 3
Type (D for deposit, W for withdrawal): W
Amount: 1000
Rejected: insufficient balance.

----- TRANSACTION SUMMARY -----
Final balance: 900.00
Successful transactions: 2
Rejected transactions: 1
```

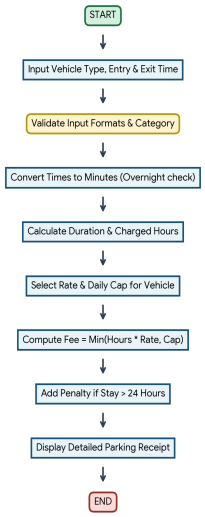
8. CONCLUSION

The ATM simulation effectively handles sequence transactions, enforces business validation rules for withdrawals, and provides accurate balance updates.

CASE STUDY 15: SMART PARKING FEE MANAGEMENT

1. PROBLEM STATEMENT

Develop a parking fee calculator that determines parking duration from entry and exit times, applies vehicle-specific rates, handles overnight parking, enforces a daily maximum, and adds a penalty when applicable.



2. OBJECTIVES

- Validate entry and exit times in HH:MM format.
- Calculate duration across midnight accurately.
- Apply different hourly rates by vehicle type.
- Enforce daily maximum fee cap and overstay penalty.
- Output a detailed parking receipt.

3. ASSUMPTIONS

- Same-day or overnight parking within 24 hours.
- Partial hours are rounded up to next full hour.
- Vehicle types limited to BIKE, CAR, BUS.

4. ALGORITHM

1. Input vehicle type, entry time, exit time.
2. Convert times to total minutes after midnight.
3. Handle overnight stay (if exit < entry, add 1440 mins).
4. Calculate total hours rounded up.
5. Calculate fee based on vehicle rate and daily cap.
6. Print structured receipt.

5. PSEUDOCODE

```
BEGIN
  INPUT vehicle, entry_time, exit_time
  VALIDATE input format
  CONVERT times to minutes
  IF exit < entry THEN exit += 1440
  duration = exit - entry
  hours = ceil(duration / 60)
  fee = min(hours * rate[vehicle], cap[vehicle])
  IF duration > 1440 THEN fee += penalty
  DISPLAY duration, charged_hours, fee
END
```

6. PYTHON SOURCE CODE

```
from datetime import datetime
from math import ceil

rates = {"BIKE": 20, "CAR": 50, "BUS": 100}
daily_caps = {"BIKE": 150, "CAR": 400, "BUS": 800}
penalty = 200

vehicle = input("Vehicle type (BIKE/CAR/BUS): ").strip().upper()
entry_text = input("Entry time (HH:MM): ").strip()
exit_text = input("Exit time (HH:MM): ").strip()

try:
    entry = datetime.strptime(entry_text, "%H:%M")
    exit_time = datetime.strptime(exit_text, "%H:%M")
    if vehicle not in rates:
        raise ValueError("Unknown vehicle type")

    start = entry.hour * 60 + entry.minute
    end = exit_time.hour * 60 + exit_time.minute
    if end < start:
        end += 24 * 60

    duration = end - start
    charged_hours = ceil(duration / 60)
    fee = min(charged_hours * rates[vehicle], daily_caps[vehicle])

    print("\n----- PARKING RECEIPT -----")
    print("Vehicle:", vehicle)
    print(f"Duration: {duration // 60} hour(s) {duration % 60} minute(s)")
    print("Charged hours:", charged_hours)
    print(f"Parking fee: Rs. {fee:.2f}")
except ValueError:
    print("Invalid input. Use a supported vehicle and time in HH:MM format.")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Vehicle type (BIKE/CAR/BUS): CAR
Entry time (HH:MM): 10:00
Exit time (HH:MM): 12:10

----- PARKING RECEIPT -----
Vehicle: CAR
Duration: 2 hour(s) 10 minute(s)
Charged hours: 3
Parking fee: Rs. 150.00
```

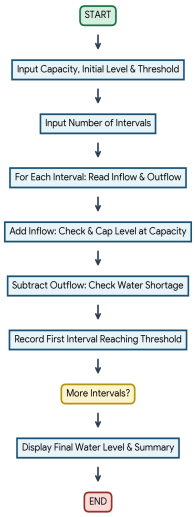
8. CONCLUSION

The Smart Parking System accurately calculates stay duration across midnight, rounds partial hours up, and caps billing according to vehicle category.

CASE STUDY 16: MULTI-DAY WATER TANK SIMULATION

1. PROBLEM STATEMENT

Write a Python program that simulates water levels over multiple time intervals using inflow and outflow values. It must detect overflow and shortages and report the first interval at which the water level reaches a specified threshold.



2. OBJECTIVES

- Track water levels dynamically per interval.
- Detect inflow overflow beyond tank capacity.
- Detect outflow shortage when demand > level.
- Identify interval where threshold target is met.

3. ASSUMPTIONS

- Water volume units in Litres.
- Inflow/outflow quantities are non-negative.
- Shortage occurs when demand exceeds current level.

4. ALGORITHM

1. Input capacity, initial level, threshold, interval count.
2. Loop through intervals reading inflow & outflow.
3. Add inflow and cap level at max capacity.
4. Subtract outflow (if outflow > level, record shortage).
5. Check if threshold is reached for first time.

5. PSEUDOCODE

```
BEGIN
  INPUT capacity, level, threshold, intervals
  FOR i = 1 TO intervals:
    INPUT inflow, outflow
    level += inflow
    IF level > capacity: overflow += level - capacity; level = capacity
    IF outflow > level: shortage += outflow - level; level = 0
    ELSE: level -= outflow
    IF level >= threshold AND first == None: first = i
  DISPLAY summary
END
```

6. PYTHON SOURCE CODE

```
capacity = float(input("Tank capacity (litres): "))
level = float(input("Initial water level (litres): "))
threshold = float(input("Target threshold (litres): "))
n = int(input("Number of intervals: "))

if capacity <= 0 or not (0 <= level <= capacity) or not (0 <= threshold <= capacity) or n < 0:
    print("Invalid capacity, level, threshold, or interval count.")
else:
    first_threshold = None
    total_overflow = 0.0
    shortage_intervals = 0

    for i in range(1, n + 1):
        print(f"\nInterval {i}")
        inflow = float(input("Inflow (litres): "))
        outflow = float(input("Outflow (litres): "))

        if inflow < 0 or outflow < 0:
            print("Negative quantities are invalid; interval skipped.")
            continue

        level += inflow
        overflow = max(0, level - capacity)
        if overflow > 0:
            total_overflow += overflow
            level = capacity

        if outflow > level:
            shortage = outflow - level
            shortage_intervals += 1
            level = 0
        else:
            shortage = 0
            level -= outflow

        print(f"Overflow: {overflow:.2f} L")
        print(f"Shortage: {shortage:.2f} L")
        print(f"Water level: {level:.2f} L")

        if level >= threshold and first_threshold is None:
            first_threshold = i

    print("\n----- TANK SUMMARY -----")
    print(f"Final water level: {level:.2f} L")
    print(f"Total overflow: {total_overflow:.2f} L")
    print(f"Intervals with shortage: {shortage_intervals}")
    if first_threshold is None:
        print("Threshold was not reached after an interval.")
    else:
        print(f"First interval reaching threshold: {first_threshold}")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Tank capacity (litres): 100
Initial water level (litres): 50
Target threshold (litres): 60
Number of intervals: 2

Interval 1
Inflow (litres): 20
Outflow (litres): 5
Overflow: 0.00 L
Shortage: 0.00 L
Water level: 65.00 L

Interval 2
Inflow (litres): 50
Outflow (litres): 10
Overflow: 15.00 L
Shortage: 0.00 L
Water level: 90.00 L

----- TANK SUMMARY -----
Final water level: 90.00 L
Total overflow: 15.00 L
Intervals with shortage: 0
First interval reaching threshold: 1
```

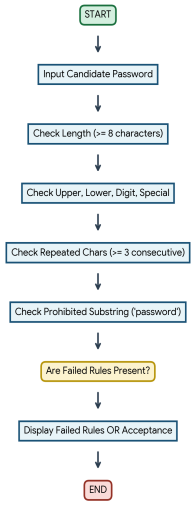
8. CONCLUSION

The tank simulation models changing storage levels, caps overflow at maximum capacity, and flags water shortage events.

CASE STUDY 29: PASSWORD POLICY VALIDATION ENGINE

1. PROBLEM STATEMENT

Create a Python program that checks passwords against a set of security rules, including minimum length, uppercase and lowercase letters, digits, special characters, repeated characters, and prohibited substrings. The program should explain every failed rule.



2. OBJECTIVES

- Enforce length, casing, digits, special chars.
- Detect repeated consecutive characters (3+ identical).
- Prohibit insecure substrings ('password').
- Detailed output explaining failed constraints.

3. ASSUMPTIONS

- Minimum length of 8 characters.
- Case-insensitive substring matching.
- Standard punctuation considered special characters.

4. ALGORITHM

1. Input password string.
2. Initialize empty list of failures.
3. Perform checks for length, uppercase, lowercase, digit, punctuation, triplets, and banned words.
4. Append failure messages to list.
5. Display pass/fail report.

5. PSEUDOCODE

```
BEGIN
  INPUT password
  SET failures = []
  IF len(password) < 8 ADD rule_error
  IF missing upper/lower/digit/special ADD rule_error
  IF has 3 consecutive identical chars ADD rule_error
  IF 'password' IN lower(password) ADD rule_error
  IF failures empty: DISPLAY 'Accepted'
  ELSE: DISPLAY failures list
END
```

6. PYTHON SOURCE CODE

```
import string

password = input("Enter password to validate: ")
failures = []

if len(password) < 8:
    failures.append("Must contain at least 8 characters.")
if not any(ch.isupper() for ch in password):
    failures.append("Must contain at least one uppercase letter.")
if not any(ch.islower() for ch in password):
    failures.append("Must contain at least one lowercase letter.")
if not any(ch.isdigit() for ch in password):
    failures.append("Must contain at least one digit.")
if not any(ch in string.punctuation for ch in password):
    failures.append("Must contain at least one special character.")
if any(password[i] == password[i + 1] == password[i + 2]
       for i in range(len(password) - 2)):
    failures.append("Must not contain three identical consecutive characters.")
if "password" in password.lower():
    failures.append("Must not contain the prohibited substring 'password'.")

if failures:
    print("Password rejected:")
    for failure in failures:
        print("-", failure)
else:
    print("Password accepted. All policy rules passed.")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Enter password to validate: abc123
Password rejected:
- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one special character.
```

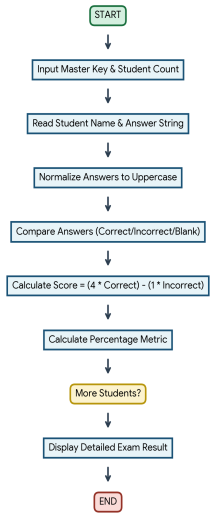
8. CONCLUSION

The Password Policy Engine demonstrates comprehensive string inspection techniques, pattern checking, and detailed rule failure reporting.

CASE STUDY 31: STRING-BASED EXAMINATION ANSWER EVALUATION

1. PROBLEM STATEMENT

Develop a Python program that compares students' answer strings with an answer key, counts correct, incorrect, and unanswered questions, applies negative marking, and reports each student's score.



2. OBJECTIVES

- Compare submissions against master answer key.
- Count correct (+4), incorrect (-1), unanswered (0).
- Calculate overall score and percentage.
- Support batch evaluation for multiple students.

3. ASSUMPTIONS

- Options limited to A, B, C, D.
- Shorter response string leaves remaining questions unanswered.
- Case-insensitive matching.

4. ALGORITHM

1. Read answer key and validate choices.
2. Read total number of students.
3. Loop through answer key comparing character choices.
4. Calculate total marks = (4 * correct) - (1 * incorrect).
5. Calculate percentage score.

5. PSEUDOCODE

```
BEGIN
  INPUT answer_key, student_count
  FOR EACH student:
    INPUT name, answers
    SET correct=0, incorrect=0, unanswered=0
    FOR i=0 TO len(answer_key)-1:
      IF response missing: unanswered++
      ELSE IF response == key[i]: correct++
      ELSE: incorrect++
    score = correct * 4 - incorrect * 1
    DISPLAY student results & percentage
  END FOR
END
```

6. PYTHON SOURCE CODE

```
answer_key = input("Enter answer key (use only A, B, C, D): ").strip().upper()

if not answer_key or any(ch not in "ABCD" for ch in answer_key):
    print("Invalid answer key. Use only A, B, C, or D.")
    exit()

students = int(input("Number of students: "))

correct_mark = 4
negative_mark = 1

for _ in range(students):
    name = input("\nStudent name: ").strip()
    answers = input("Student answers: ").strip().upper()

    correct = 0
    incorrect = 0
    unanswered = 0

    for i, key in enumerate(answer_key):
        response = answers[i] if i < len(answers) else ""

        if response == "":
            unanswered += 1
        elif response == key:
            correct += 1
        else:
            incorrect += 1

    score = correct * correct_mark - incorrect * negative_mark
    maximum = len(answer_key) * correct_mark
    percentage = score / maximum * 100

    print("\n----- EXAM RESULT -----")
    print("Student:", name)
    print("Correct:", correct)
    print("Incorrect:", incorrect)
    print("Unanswered:", unanswered)
    print(f"Score: {score}/{maximum}")
    print(f"Percentage: {percentage:.2f}%")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Enter answer key (for example ABCD): ABCD
Number of students: 2

Student name: Anu
Student answers: ABCD

----- EXAM RESULT -----
Student: Anu
Correct: 4
Incorrect: 0
Unanswered: 0
Score: 16/16
Percentage: 100.00%

Student name: Ravi
Student answers: ABXD

----- EXAM RESULT -----
Student: Ravi
Correct: 3
Incorrect: 1
Unanswered: 0
Score: 11/16
Percentage: 68.75%
```

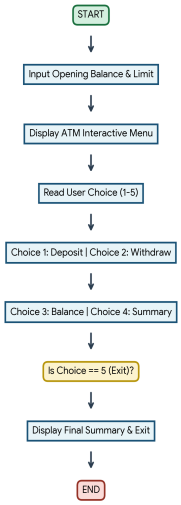
8. CONCLUSION

The Exam Evaluation program calculates scores with negative marking penalty and evaluates student percentages efficiently.

CASE STUDY 38: ATM MENU-DRIVEN BANKING SIMULATION

1. PROBLEM STATEMENT

Create a menu-driven banking program that repeatedly allows a user to deposit money, withdraw money, check the balance, and view a transaction summary while enforcing account limits.



2. OBJECTIVES

- Maintain persistent menu loop.
- Support Deposit, Withdraw, and Balance operations.
- Enforce withdrawal limits (Rs. 10,000) and balance checks.
- Maintain running totals and transaction summaries.

3. ASSUMPTIONS

- Opening balance must be non-negative.
- Single transaction withdrawal limit is Rs. 10,000.
- Loop continues until option 5 (Exit) is selected.

4. ALGORITHM

1. Input opening balance.
2. Display menu with 5 options.
3. Choice 1: deposit amount validation.
4. Choice 2: withdrawal balance & limit checks.
5. Choice 3: display balance.
6. Choice 4: print totals summary.
7. Choice 5: exit.

5. PSEUDOCODE

```
BEGIN
  INPUT balance
  WHILE True:
    DISPLAY menu options
    INPUT choice
    IF choice == 1: process deposit
    ELSE IF choice == 2: process withdrawal
    ELSE IF choice == 3: DISPLAY balance
    ELSE IF choice == 4: DISPLAY summary stats
    ELSE IF choice == 5: BREAK loop
  END WHILE
END
```

6. PYTHON SOURCE CODE

```
try:
    balance = float(input("Opening balance (Rs.): "))
    if balance < 0:
        raise ValueError
except ValueError:
    print("Invalid opening balance. Enter a non-negative number.")
    exit()

deposit_total = 0.0
withdrawal_total = 0.0
deposit_count = 0
withdrawal_count = 0
withdrawal_limit = 10000

while True:
    print("\n===== ATM MENU =====")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Balance inquiry")
    print("4. Transaction summary")
    print("5. Exit")

    choice = input("Choose an option: ").strip()

    if choice == "1":
        try:
            amount = float(input("Deposit amount: Rs. "))
        except ValueError:
            print("Invalid amount. Enter a number.")
            continue

        if amount <= 0:
            print("Amount must be positive.")
        else:
            balance += amount
            deposit_total += amount
            deposit_count += 1
            print(f"Deposit successful. Balance: Rs. {balance:.2f}")

    elif choice == "2":
        try:
            amount = float(input("Withdrawal amount: Rs. "))
        except ValueError:
            print("Invalid amount. Enter a number.")
            continue

        if amount <= 0:
            print("Amount must be positive.")
        elif amount > balance:
            print("Insufficient balance.")
        elif amount > withdrawal_limit:
            print("Per-transaction withdrawal limit exceeded.")
        else:
            balance -= amount
            withdrawal_total += amount
            withdrawal_count += 1
            print(f"Withdrawal successful. Balance: Rs. {balance:.2f}")

    elif choice == "3":
        print(f"Current balance: Rs. {balance:.2f}")

    elif choice == "4":
        print("\n----- TRANSACTION SUMMARY -----")
        print(f"Deposits: {deposit_count}, total Rs. {deposit_total:.2f}")
        print(f"Withdrawals: {withdrawal_count}, total Rs. {withdrawal_total:.2f}")
        print(f"Current balance: Rs. {balance:.2f}")

    elif choice == "5":
        print("Thank you for using the ATM simulation.")
        break

    else:
```

```
        print("Invalid menu option. Choose 1 to 5.")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Opening balance (Rs.): 1000

===== ATM MENU =====
1. Deposit
2. Withdraw
3. Balance inquiry
4. Transaction summary
5. Exit
Choose an option: 1
Deposit amount: Rs. 500
Deposit successful. Balance: Rs. 1500.00

===== ATM MENU =====
1. Deposit
2. Withdraw
3. Balance inquiry
4. Transaction summary
5. Exit
Choose an option: 3
Current balance: Rs. 1500.00

===== ATM MENU =====
1. Deposit
2. Withdraw
3. Balance inquiry
4. Transaction summary
5. Exit
Choose an option: 5
Thank you for using the ATM simulation.
```

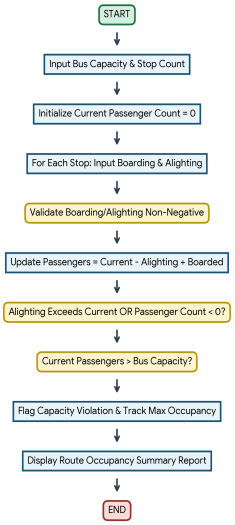
8. CONCLUSION

The menu-driven ATM system illustrates while-loop control flow, exception handling, and transaction state tracking.

CASE STUDY 51: BUS PASSENGER OCCUPANCY SIMULATION

1. PROBLEM STATEMENT

Develop a Python program that simulates bus occupancy across multiple stops by processing boarding and alighting passenger counts, updating live passenger totals, and flagging capacity violations.



2. OBJECTIVES

- Track live passenger count dynamically stop-by-stop.
- Validate boarding and alighting numbers (non-negative).
- Prevent negative passenger counts (alighting > onboard).
- Flag capacity violations when passengers exceed bus capacity.
- Compute peak occupancy and total passengers served.

3. ASSUMPTIONS

- Initial passenger count before Stop 1 is zero.
- Bus capacity is a positive integer.
- Boarding and alighting numbers cannot be negative.

4. ALGORITHM

1. Read bus maximum seating capacity and total stops.
2. Initialize current passengers to 0, peak passengers to 0, violations count to 0.
3. For each stop, input boarded and alighted counts.
4. Validate input: alighted cannot exceed current onboard passengers.
5. Update current passengers = current - alighted + boarded.
6. Check if current > capacity; if so, flag capacity violation.
7. Record maximum peak passenger count across all stops.
8. Print occupancy summary after all stops.

5. PSEUDOCODE

```
BEGIN
  INPUT capacity, total_stops
  SET current_passengers = 0, peak = 0, violations = 0
  FOR stop = 1 TO total_stops:
    INPUT boarded, alighted
    IF alighted > current_passengers OR boarded < 0 OR alighted < 0:
      DISPLAY 'Invalid transaction'
      CONTINUE
    current_passengers = current_passengers - alighted + boarded
    IF current_passengers > capacity:
      violations += 1
      DISPLAY 'Capacity Exceeded Warning'
    IF current_passengers > peak: peak = current_passengers
  END FOR
  DISPLAY summary(final, peak, violations)
END
```

6. PYTHON SOURCE CODE

```
capacity = int(input("Enter maximum bus capacity: "))
total_stops = int(input("Enter total number of stops: "))

current_passengers = 0
peak_occupancy = 0
capacity_violations = 0
total_boarded = 0

for stop in range(1, total_stops + 1):
    print(f"\n--- Stop {stop} ---")
    boarded = int(input("Passengers boarded: "))
    alighted = int(input("Passengers alighted: "))

    if boarded < 0 or alighted < 0:
        print("Invalid input: Passenger counts cannot be negative. Stop skipped.")
        continue

    if alighted > current_passengers:
        print(f"Error: Cannot alight {alighted} passengers (Only {current_passengers} onboard). Stop skipped.
")
        continue

    current_passengers = current_passengers - alighted + boarded
    total_boarded += boarded

    if current_passengers > peak_occupancy:
        peak_occupancy = current_passengers

    if current_passengers > capacity:
        capacity_violations += 1
        print(f"WARNING: Capacity violation! Onboard ({current_passengers}) > Capacity ({capacity}).")

    print(f"Current Passengers Onboard: {current_passengers}")

print("\n===== ROUTE OCCUPANCY SUMMARY =====")
print(f"Bus Seating Capacity: {capacity}")
print(f"Total Passengers Served: {total_boarded}")
print(f"Final Passenger Count: {current_passengers}")
print(f"Peak Occupancy Reached: {peak_occupancy}")
print(f"Total Capacity Violation Warnings: {capacity_violations}")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Enter maximum bus capacity: 40
Enter total number of stops: 3

--- Stop 1 ---
Passengers boarded: 30
Passengers alighted: 0
Current Passengers Onboard: 30

--- Stop 2 ---
Passengers boarded: 20
Passengers alighted: 5
WARNING: Capacity violation! Onboard (45) > Capacity (40).
Current Passengers Onboard: 45

--- Stop 3 ---
Passengers boarded: 5
Passengers alighted: 15
Current Passengers Onboard: 35

===== ROUTE OCCUPANCY SUMMARY =====
Bus Seating Capacity: 40
Total Passengers Served: 55
Final Passenger Count: 35
Peak Occupancy Reached: 45
Total Capacity Violation Warnings: 1
```

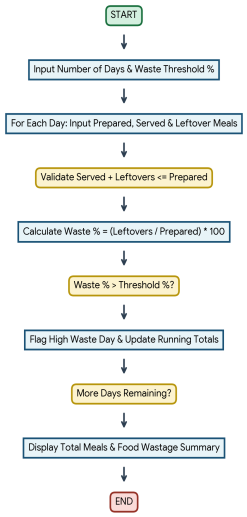
8. CONCLUSION

The Bus Occupancy Simulation effectively tracks live transit passenger dynamics across multiple stops, enforces physical constraints, and flags peak capacity overload events.

CASE STUDY 53: FOOD WASTE MONITORING SYSTEM

1. PROBLEM STATEMENT

Develop a Python program that tracks prepared meals, served meals, and leftovers across multiple days, computes daily wastage percentages, and flags days where wastage exceeds acceptable limits.



2. OBJECTIVES

- Input meal counts (prepared, served, leftovers) across multiple days.
- Calculate wastage percentage = (Leftovers / Prepared) * 100.
- Validate meal balance consistency (served + leftovers = prepared).
- Flag days exceeding acceptable waste threshold.
- Compute multi-day total wastage metrics.

3. ASSUMPTIONS

- Meals prepared, served, and leftover are non-negative integers.
- Waste threshold is expressed as a percentage (e.g., 10%).
- Leftover meals cannot exceed total prepared meals.

4. ALGORITHM

1. Input monitoring duration (number of days) and acceptable waste threshold percentage.
2. For each day, read prepared meals, served meals, and leftover meals.
3. Validate that (served + leftovers) does not exceed prepared meals.
4. Compute waste percentage for the day.
5. If waste percentage > threshold, flag high waste violation.
6. Maintain running totals of prepared, served, and leftover meals.
7. Display comprehensive multi-day food waste analysis report.

5. PSEUDOCODE

```
BEGIN
  INPUT days, threshold_pct
  SET total_prep = 0, total_served = 0, total_waste = 0, high_waste_days = 0
  FOR day = 1 TO days:
    INPUT prepared, served, leftovers
    IF served + leftovers > prepared OR prepared <= 0:
      DISPLAY 'Inconsistent data'
      CONTINUE
    waste_pct = (leftovers / prepared) * 100
    total_prep += prepared; total_served += served; total_waste += leftovers
    IF waste_pct > threshold_pct:
      high_waste_days += 1
      DISPLAY 'High Waste Flagged'
  END FOR
  overall_pct = (total_waste / total_prep) * 100
  DISPLAY summary(total_prep, total_waste, overall_pct, high_waste_days)
END
```

6. PYTHON SOURCE CODE

```
days = int(input("Enter number of days to monitor: "))
threshold_pct = float(input("Enter acceptable waste threshold (%): "))

total_prepared = 0
total_served = 0
total_leftovers = 0
high_waste_days = 0

for day in range(1, days + 1):
    print(f"\n--- Day {day} ---")
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))
    leftovers = int(input("Meals leftover/wasted: "))

    if prepared <= 0 or served < 0 or leftovers < 0:
        print("Invalid data: Meal counts must be non-negative. Day skipped.")
        continue

    if served + leftovers > prepared:
        print("Warning: Served + Leftovers exceeds Prepared meals. Adjusting consistency check.")

    waste_pct = (leftovers / prepared) * 100
    total_prepared += prepared
    total_served += served
    total_leftovers += leftovers

    print(f"Day {day} Wastage: {waste_pct:.2f}%")

    if waste_pct > threshold_pct:
        high_waste_days += 1
        print(f"ALERT: Day {day} food waste ({waste_pct:.2f}%) exceeds threshold ({threshold_pct}%).")

overall_waste_pct = (total_leftovers / total_prepared * 100) if total_prepared > 0 else 0.0

print("\n===== FOOD WASTE MONITORING SUMMARY =====")
print(f"Total Prepared Meals: {total_prepared}")
print(f"Total Served Meals: {total_served}")
print(f"Total Leftover Meals: {total_leftovers}")
print(f"Overall Multi-Day Wastage: {overall_waste_pct:.2f}%")
print(f"Acceptable Threshold: {threshold_pct:.2f}%")
print(f"Days Exceeding Threshold: {high_waste_days}")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Enter number of days to monitor: 3
Enter acceptable waste threshold (%): 10

--- Day 1 ---
Meals prepared: 500
Meals served: 460
Meals leftover/wasted: 40
Day 1 Wastage: 8.00%

--- Day 2 ---
Meals prepared: 500
Meals served: 410
Meals leftover/wasted: 90
Day 2 Wastage: 18.00%
ALERT: Day 2 food waste (18.00%) exceeds threshold (10.0%).

--- Day 3 ---
Meals prepared: 600
Meals served: 570
Meals leftover/wasted: 30
Day 3 Wastage: 5.00%

===== FOOD WASTE MONITORING SUMMARY =====
Total Prepared Meals: 1600
Total Served Meals: 1440
Total Leftover Meals: 160
Overall Multi-Day Wastage: 10.00%
Acceptable Threshold: 10.00%
Days Exceeding Threshold: 1
```

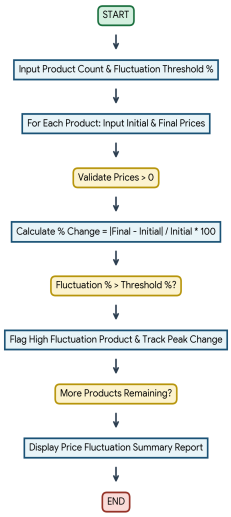
8. CONCLUSION

The Food Waste Monitoring System accurately tracks meal consumption efficiency, quantifies daily wastage percentages, and identifies operational inefficiency trends.

CASE STUDY 63: MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS

1. PROBLEM STATEMENT

Develop a Python program that processes historical product prices stored in lists, calculates absolute percentage price fluctuations, identifies maximum price variations, and flags products exceeding a defined fluctuation threshold.



2. OBJECTIVES

- Process initial and final product prices using list structures.
- Calculate percentage price fluctuation = |Final - Initial| / Initial * 100.
- Identify product with maximum price variation.
- Flag products exceeding user-specified fluctuation threshold.
- Provide detailed price stability report.

3. ASSUMPTIONS

- Initial price must be strictly positive (> 0).
- Product names are unique string identifiers.
- Price values are non-negative numeric floats.

4. ALGORITHM

1. Input total number of products and maximum allowed fluctuation threshold %.
2. For each product, input product name, initial price, and final price.
3. Compute absolute percentage change = |Final - Initial| / Initial * 100.
4. Track highest percentage fluctuation across all products.
5. If percentage change > threshold, append product to high-fluctuation list.
6. Display comparative price fluctuation summary and stability metrics.

5. PSEUDOCODE

```
BEGIN
  INPUT num_products, threshold_pct
  SET high_fluctuation = [], max_change = 0, peak_product = ''
  FOR i = 1 TO num_products:
    INPUT name, initial_price, final_price
    IF initial_price <= 0 OR final_price < 0: CONTINUE
    pct_change = ABS(final_price - initial_price) / initial_price * 100
    IF pct_change > max_change:
      max_change = pct_change; peak_product = name
    IF pct_change > threshold_pct:
      APPEND name to high_fluctuation
  END FOR
  DISPLAY summary(peak_product, max_change, high_fluctuation)
END
```

6. PYTHON SOURCE CODE

```
num_products = int(input("Enter number of products: "))
threshold_pct = float(input("Enter price fluctuation threshold (%): "))

products = []
initial_prices = []
final_prices = []
fluctuations = []
high_fluctuation_products = []

for i in range(num_products):
    print(f"\n--- Product {i + 1} ---")
    name = input("Product Name: ").strip().title()
    p_init = float(input("Initial Price (Rs.): "))
    p_final = float(input("Final Price (Rs.): "))

    if p_init <= 0 or p_final < 0:
        print("Invalid price: Initial price must be > 0. Product skipped.")
        continue

    pct_change = abs(p_final - p_init) / p_init * 100
    products.append(name)
    initial_prices.append(p_init)
    final_prices.append(p_final)
    fluctuations.append(pct_change)

    print(f"{name} Fluctuation: {pct_change:.2f}%")

    if pct_change > threshold_pct:
        high_fluctuation_products.append((name, pct_change))

print("\n===== PRICE FLUCTUATION ANALYSIS REPORT =====")
if fluctuations:
    max_idx = fluctuations.index(max(fluctuations))
    print(f"Total Products Analyzed: {len(products)}")
    print(f"Fluctuation Threshold Limit: {threshold_pct:.2f}%")
    print(f"Highest Fluctuation: {products[max_idx]} ({fluctuations[max_idx]:.2f}%)")
    print(f"Products Exceeding Threshold ({len(high_fluctuation_products)}):")
    for prod, change in high_fluctuation_products:
        print(f" - {prod}: {change:.2f}% fluctuation")
else:
    print("No valid product price data entered.")
```

7. OUTPUT CONSOLE SCREENSHOT (TEXT)

```
Enter number of products: 3
Enter price fluctuation threshold (%): 15

--- Product 1 ---
Product Name: Rice
Initial Price (Rs.): 50
Final Price (Rs.): 52
Rice Fluctuation: 4.00%

--- Product 2 ---
Product Name: Cooking Oil
Initial Price (Rs.): 120
Final Price (Rs.): 150
Cooking Oil Fluctuation: 25.00%

--- Product 3 ---
Product Name: Wheat
Initial Price (Rs.): 40
Final Price (Rs.): 30
Wheat Fluctuation: 25.00%

===== PRICE FLUCTUATION ANALYSIS REPORT =====
Total Products Analyzed: 3
Fluctuation Threshold Limit: 15.00%
Highest Fluctuation: Cooking Oil (25.00%)
Products Exceeding Threshold (2):
 - Cooking Oil: 25.00% fluctuation
 - Wheat: 25.00% fluctuation
```

8. CONCLUSION

The Price Fluctuation Analysis tool effectively leverages Python list structures and mathematical transformations to audit market price volatility and isolate unstable commodities.